

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Course Code : CSE 3102
Course Name : Database Systems Sessional
Lab No. : 10
Task Name : Exercise on SQL Queries
Date of Lab : 16 June 2025

Submitted by
Md. Raiyan Haque Ashwas
Roll: 2103063
Section: B
Series: 21

Submitted to
Md. Azmain Yakin Srizon
Assistant Professor
Department of
Computer Science & Engineering,
Rajshahi University of Engineering &
Technology, Rajshahi.

Date of Submission : 18 June 2025

Q1. Create a database called “MovieAnalyticsDB”. Write the corresponding query for this in your report.

Query:

```
CREATE DATABASE MovieAnalyticsDB;
```

Output:

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0007 seconds.)

```
CREATE DATABASE MovieAnalyticsDB;
```

Q2. Create a table titled “movies” with the following constraints. Write the corresponding query for this in your report.

- movie_id (INT, PK, AUTO_INCREMENT, NOT NULL)
- title (VARCHAR(100), NOT NULL)
- release_year (YEAR, NOT NULL)
- genre (VARCHAR(50), NOT NULL)

Query:

```
CREATE TABLE movies(  
    movie_id INT PRIMARY KEY AUTO_INCREMENT NOT NULL,  
    title VARCHAR(100) NOT NULL,  
    release_year YEAR NOT NULL,  
    genre VARCHAR(50) NOT NULL  
);
```

Output:

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

```
CREATE TABLE movies( movie_id INT PRIMARY KEY AUTO_INCREMENT NOT NULL, title VARCHAR(100)  
NOT NULL, release_year YEAR NOT NULL, genre VARCHAR(50) NOT NULL );
```

Q3. Create a table titled “actors” with the following constraints. Write the corresponding query for this in your report.

- actor_id (INT, PK, AUTO_INCREMENT, NOT NULL)
- actor_name (VARCHAR(100), NOT NULL)
- nationality (VARCHAR(50)) - date_of_birth (DATE, NOT NULL)

Query:

```
CREATE TABLE actors(
    actor_id INT PRIMARY KEY AUTO_INCREMENT NOT NULL,
    actor_name VARCHAR(100) NOT NULL,
    nationality VARCHAR(50),
    date_of_birth DATE NOT NULL
);
```

Output:

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE TABLE actors( actor_id INT PRIMARY KEY AUTO_INCREMENT NOT NULL, actor_name
VARCHAR(100) NOT NULL, nationality VARCHAR(50), date_of_birth DATE NOT NULL );
```

Q4. Create a table titled “Roles” with the following constraints. Write the corresponding query for this in your report.

- role_id (INT, PK, AUTO_INCREMENT, NOT NULL)
- actor_id (INT, FK, NOT NULL)
- movie_id (INT, FK, NOT NULL)
- character_name (VARCHAR(100))
- screen_time_minutes (INT DEFAULT 0)

Query:

```
CREATE TABLE roles (
    role_id INT PRIMARY KEY AUTO_INCREMENT NOT NULL,
    actor_id INT NOT NULL,
    movie_id INT NOT NULL,
    character_name VARCHAR(100),
    screen_time_minutes INT DEFAULT 0,
    FOREIGN KEY (actor_id) REFERENCES actors(actor_id),
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id)
);
```

Output:

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
CREATE TABLE roles ( role_id INT PRIMARY KEY AUTO_INCREMENT NOT NULL, actor_id INT NOT
NULL, movie_id INT NOT NULL, character_name VARCHAR(100), screen_time_minutes INT DEFAULT
0, FOREIGN KEY (actor_id) REFERENCES actors(actor_id), FOREIGN KEY (movie_id) REFERENCES
movies(movie_id) );
```

Q5. Populate the table “movies” with the following data. Write the corresponding insertion query for this in your report.

title	release_year	genre
Inception	2010	Sci-Fi
The Dark Knight	2008	Action
Interstellar	2014	Sci-Fi
Titanic	1997	Romance
Joker	2019	Drama

Query:

```
INSERT INTO movies (title, release_year, genre) VALUES
('Inception', '2010', 'Sci-Fi'),
('The Dark Knight', '2008', 'Action'),
('Interstellar', '2014', 'Sci-Fi'),
('Titanic', '1997', 'Romance'),
('Joker', '2019', 'Drama');
```

Output:

✓ 5 rows inserted.

Inserted row id: 5 (Query took 0.0024 seconds.)

```
INSERT INTO movies (title, release_year, genre) VALUES ('Inception', '2010', 'Sci-Fi'),
('The Dark Knight', '2008', 'Action'), ('Interstellar', '2014', 'Sci-Fi'), ('Titanic',
'1997', 'Romance'), ('Joker', '2019', 'Drama');
```

Q6. Populate the table “actors” with the following data. Write the corresponding insertion query for this in your report.

actor_name	nationality	date_of_birth
Leonardo DiCaprio	American	1974-11-11
Christian Bale	British	1974-01-30
Anne Hathaway	American	1982-11-12
Kate Winslet	British	1975-10-05
Joaquin Phoenix	American	1974-10-28

Query:

```
INSERT INTO actors (actor_name, nationality, date_of_birth) VALUES
('Leonardo DiCaprio', 'American', '1974-11-11'),
('Christian Bale', 'British', '1974-01-30'),
('Anne Hathaway', 'American', '1982-11-12'),
('Kate Winslet', 'British', '1975-10-05'),
('Joaquin Phoenix', 'American', '1974-10-28');
```

Output:

✓ 5 rows inserted.

Inserted row id: 5 (Query took 0.0023 seconds.)

```
INSERT INTO actors (actor_name, nationality, date_of_birth) VALUES ('Leonardo DiCaprio',
'American', '1974-11-11'), ('Christian Bale', 'British', '1974-01-30'), ('Anne Hathaway',
'American', '1982-11-12'), ('Kate Winslet', 'British', '1975-10-05'), ('Joaquin Phoenix',
'American', '1974-10-28');
```

Q7. Populate the table “Roles” with the following data. Write the corresponding insertion query for this in your report.

actor_id	movie_id	character_name	screen_time_minutes
1	1	Dom Cobb	90
1	4	Jack Dawson	80
2	2	Bruce Wayne	110
3	3	Brand	75
4	4	Rose DeWitt Bukater	85
5	5	Arthur Fleck	115

Query:

```
INSERT INTO Roles (actor_id, movie_id, character_name, screen_time_minutes)
VALUES
(1, 1, 'Dom Cobb', 90),
(1, 4, 'Jack Dawson', 80),
(2, 2, 'Bruce Wayne', 110),
(3, 3, 'Brand', 75),
(4, 4, 'Rose DeWitt Bukater', 85),
(5, 5, 'Arthur Fleck', 115);
```

Output:

✓ 6 rows inserted.

Inserted row id: 6 (Query took 0.0003 seconds.)

```
INSERT INTO Roles (actor_id, movie_id, character_name, screen_time_minutes) VALUES (1, 1,
'Dom Cobb', 90), (1, 4, 'Jack Dawson', 80), (2, 2, 'Bruce Wayne', 110), (3, 3, 'Brand',
75), (4, 4, 'Rose DeWitt Bukater', 85), (5, 5, 'Arthur Fleck', 115);
```

Q8. Random Queries:

- a) Display all movies.
- b) List all actors born after 1980.
- c) Find all roles with screen time more than 100 minutes.
- d) Count the number of movies in each genre.
- e) Show each actor with total screen time.
- f) Find actors who acted in more than one movie.
- g) Find movies where Leonardo DiCaprio acted.
- h) Show average screen time per movie.
- i) List top 3 actors with highest total screen time.
- j) List genres along with number of unique actors participated.

Solution:

Sl No.	Query	Output																								
a	<code>SELECT * FROM movies</code>	<table><tr><th>movie_id</th><th>title</th><th>release_year</th><th>genre</th></tr><tr><td>1</td><td>Inception</td><td>2010</td><td>Sci-Fi</td></tr><tr><td>2</td><td>The Dark Knight</td><td>2008</td><td>Action</td></tr><tr><td>3</td><td>Interstellar</td><td>2014</td><td>Sci-Fi</td></tr><tr><td>4</td><td>Titanic</td><td>1997</td><td>Romance</td></tr><tr><td>5</td><td>Joker</td><td>2019</td><td>Drama</td></tr></table>	movie_id	title	release_year	genre	1	Inception	2010	Sci-Fi	2	The Dark Knight	2008	Action	3	Interstellar	2014	Sci-Fi	4	Titanic	1997	Romance	5	Joker	2019	Drama
movie_id	title	release_year	genre																							
1	Inception	2010	Sci-Fi																							
2	The Dark Knight	2008	Action																							
3	Interstellar	2014	Sci-Fi																							
4	Titanic	1997	Romance																							
5	Joker	2019	Drama																							
b	<code>SELECT * FROM actors WHERE date_of_birth > '1980-01-01';</code>	<table><tr><th>actor_id</th><th>actor_name</th><th>nationality</th><th>date_of_birth</th></tr><tr><td>3</td><td>Anne Hathaway</td><td>American</td><td>1982-11-12</td></tr></table>	actor_id	actor_name	nationality	date_of_birth	3	Anne Hathaway	American	1982-11-12																
actor_id	actor_name	nationality	date_of_birth																							
3	Anne Hathaway	American	1982-11-12																							
c	<code>SELECT * FROM roles WHERE screen_time_minutes > 100;</code>	<table><tr><th>role_id</th><th>actor_id</th><th>movie_id</th><th>character_name</th><th>screen_time_minutes</th></tr><tr><td>3</td><td>2</td><td>2</td><td>Bruce Wayne</td><td>110</td></tr><tr><td>6</td><td>5</td><td>5</td><td>Arthur FLeck</td><td>115</td></tr></table>	role_id	actor_id	movie_id	character_name	screen_time_minutes	3	2	2	Bruce Wayne	110	6	5	5	Arthur FLeck	115									
role_id	actor_id	movie_id	character_name	screen_time_minutes																						
3	2	2	Bruce Wayne	110																						
6	5	5	Arthur FLeck	115																						
d	<code>SELECT genre, COUNT(*) AS movie_count FROM movies GROUP BY genre;</code>	<table><tr><th>genre</th><th>movie_count</th></tr><tr><td>Action</td><td>1</td></tr><tr><td>Drama</td><td>1</td></tr><tr><td>Romance</td><td>1</td></tr><tr><td>Sci-Fi</td><td>2</td></tr></table>	genre	movie_count	Action	1	Drama	1	Romance	1	Sci-Fi	2														
genre	movie_count																									
Action	1																									
Drama	1																									
Romance	1																									
Sci-Fi	2																									
e	<code>SELECT actors.actor_name, SUM(roles.screen_time_minutes) AS total_screen_time FROM roles JOIN actors ON roles.actor_id = actors.actor_id GROUP BY actors.actor_name;</code>	<table><tr><th>actor_name</th><th>total_screen_time</th></tr><tr><td>Anne Hathaway</td><td>75</td></tr><tr><td>Christian Bale</td><td>110</td></tr><tr><td>Joaquin Phoenix</td><td>115</td></tr><tr><td>Kate Winslet</td><td>85</td></tr><tr><td>Leonardo DiCaprio</td><td>170</td></tr></table>	actor_name	total_screen_time	Anne Hathaway	75	Christian Bale	110	Joaquin Phoenix	115	Kate Winslet	85	Leonardo DiCaprio	170												
actor_name	total_screen_time																									
Anne Hathaway	75																									
Christian Bale	110																									
Joaquin Phoenix	115																									
Kate Winslet	85																									
Leonardo DiCaprio	170																									
f	<code>SELECT actors.actor_name, COUNT(DISTINCT roles.movie_id) AS movie_count FROM roles JOIN actors ON roles.actor_id = actors.actor_id GROUP BY actors.actor_name HAVING movie count > 1;</code>	<table><tr><th>actor_name</th><th>movie_count</th></tr><tr><td>Leonardo DiCaprio</td><td>2</td></tr></table>	actor_name	movie_count	Leonardo DiCaprio	2																				
actor_name	movie_count																									
Leonardo DiCaprio	2																									

g	<pre>SELECT movies.title FROM roles JOIN movies ON roles.movie_id = movies.movie_id JOIN actors ON roles.actor_id = actors.actor_id WHERE actors.actor_name = 'Leonardo DiCaprio';</pre>	<table><tr><th>title</th></tr><tr><td>Inception</td></tr><tr><td>Titanic</td></tr></table>	title	Inception	Titanic									
title														
Inception														
Titanic														
h	<pre>SELECT movies.title, AVG(roles.screen_time_minutes) AS avg_screen_time FROM roles JOIN movies ON roles.movie_id = movies.movie_id GROUP BY movies.title;</pre>	<table><tr><th>title</th><th>avg_screen_time</th></tr><tr><td>Inception</td><td>90.0000</td></tr><tr><td>Interstellar</td><td>75.0000</td></tr><tr><td>Joker</td><td>115.0000</td></tr><tr><td>The Dark Knight</td><td>110.0000</td></tr><tr><td>Titanic</td><td>82.5000</td></tr></table>	title	avg_screen_time	Inception	90.0000	Interstellar	75.0000	Joker	115.0000	The Dark Knight	110.0000	Titanic	82.5000
title	avg_screen_time													
Inception	90.0000													
Interstellar	75.0000													
Joker	115.0000													
The Dark Knight	110.0000													
Titanic	82.5000													
i	<pre>SELECT actors.actor_name, SUM(roles.screen_time_minutes) AS total_screen_time FROM roles JOIN actors ON roles.actor_id = actors.actor_id GROUP BY actors.actor_name ORDER BY total_screen_time DESC LIMIT 3;</pre>	<table><tr><th>actor_name</th><th>total_screen_time</th><th>▼ 1</th></tr><tr><td>Leonardo DiCaprio</td><td>170</td><td></td></tr><tr><td>Joaquin Phoenix</td><td>115</td><td></td></tr><tr><td>Christian Bale</td><td>110</td><td></td></tr></table>	actor_name	total_screen_time	▼ 1	Leonardo DiCaprio	170		Joaquin Phoenix	115		Christian Bale	110	
actor_name	total_screen_time	▼ 1												
Leonardo DiCaprio	170													
Joaquin Phoenix	115													
Christian Bale	110													
j	<pre>SELECT movies.genre, COUNT(DISTINCT roles.actor_id) AS unique_actors FROM roles JOIN movies ON roles.movie_id = movies.movie_id GROUP BY movies.genre;</pre>	<table><tr><th>genre</th><th>unique_actors</th></tr><tr><td>Action</td><td>1</td></tr><tr><td>Drama</td><td>1</td></tr><tr><td>Romance</td><td>2</td></tr><tr><td>Sci-Fi</td><td>2</td></tr></table>	genre	unique_actors	Action	1	Drama	1	Romance	2	Sci-Fi	2		
genre	unique_actors													
Action	1													
Drama	1													
Romance	2													
Sci-Fi	2													